RetailPulse Analytics

Project overview, important topics, and system flows

RetailPulse is a multi-tenant retail management and analytics application. It lets each company manage its catalog, stock, sales, alerts, and analytics while keeping company data isolated.

Technology at a glance

- **Frontend: React, Vite, Material-UI, TanStack Query, Axios,** Recharts.
- **Backend: FastAPI, SQLAlchemy ORM, Pydantic schemas, JWT authentication, and** Alembic-compatible configuration.
- **Cross-cutting concerns: role access control, automatic token refresh, audit** logging, inventory movements, and notifications.

Reading map

Purpose

Start with the architecture and access model, then follow the sales and inventory flows. The analytics section explains how operational data becomes dashboard insights and exports.

1. System architecture

Browser client and API server communicate over JSON. The API owns authorization and data access.

React pages and reusable UI components

API client adds JWT access token; interceptor refreshes expired tokens

FastAPI routers validate requests and enforce role dependencies

Services apply business rules, write audit events, and coordinate modules

Repositories / SQLAlchemy models read and write company-scoped database records

Frontend responsibilities

- **Routes:** login, registration, dashboard, analytics, categories, products, sales, inventory, and profile.
- **AuthContext loads /auth/me at startup, exposes the user, login, logout, and refresh-profile actions.**
- **ProtectedRoute redirects unauthenticated visitors to login. RoleGuard hides** restricted screens or actions.
- **TanStack Query fetches and caches server data; the analytics dashboard refreshes** itself every 30 seconds.

Backend layering

- **api/:** HTTP endpoints, query parameters, request metadata, and FastAPI dependency injection.
- **services/:** domain behavior such as token rotation, price calculations, stock movements, and report export.
- **repositories/:** filtered database queries and aggregate analytics queries.
- **models/ and schemas/:** persistent table definitions and validated request/response contracts.

2. Tenant isolation and access control

Tenant boundary

Every user belongs to a Company. Protected endpoints derive company_id from the authenticated JWT user rather than trusting a client-provided company ID. Products, categories, sales, inventory, notifications, and audit events are queried within that company scope.

Roles

- **COMPANY_ADMIN: full operational control. Can manage categories and products,** adjust stock, set reorder levels, and view analytics.
- **ANALYST: can browse products and inventory, create/manage sales, view** notifications, and use analytics. Cannot alter catalog or manual stock settings.
- **SUPER_ADMIN: dependency helper grants this role access to every role-restricted** endpoint.
- **VIEWER exists as a role enum but is not granted access by the current feature** routers.

Authentication flow

Company registration creates the Company and its first COMPANY_ADMIN user.

Login validates the password and active status, then issues access and refresh JWTs.

Frontend stores both tokens in localStorage and sends the access token as Bearer authorization.

On a 401, Axios queues requests, calls /auth/refresh once, rotates tokens, and retries queued requests.

Logout revokes the submitted refresh token; password change revokes all refresh tokens for the user.

3. Core data model

The model is centered on a company-owned retail catalog and the transactions that change stock.

Main relationships

Company -> Users, Categories, Products, Sales, Inventory, Notifications, Audit Logs

Category -> Products (a company cannot repeat a category name)

Product -> one Inventory record and many Sale Items (SKU is unique within a company)

Sale -> many Sale Items (invoice number is unique within a company)

Inventory -> many immutable Inventory Movements

Important business fields

- **Product: category, SKU, brand, selling and cost price, stock_quantity,** low_stock_threshold, unit, and active/inactive status.
- **Inventory: current, reserved, and available stock; reorder level; calculated** status: IN_STOCK, LOW_STOCK, or OUT_OF_STOCK.
- **Sale: invoice, customer, date, channel, payment method, subtotal, discount, tax,** total, and individual item lines.
- **Operational traceability: AuditLog stores who did what, when, where (IP/browser),** and optionally the target name. InventoryMovement records every quantity change.

4. Catalog and inventory flows

Catalog lifecycle

Admin creates a category; duplicate names are rejected within the company.

Admin creates a product with its category, SKU, prices, opening stock, threshold, and status.

Product service keeps a matching inventory record and synchronizes reorder settings.

Direct product stock edits are represented as a manual inventory movement.

Category/product mutations are audit logged; products can be activated or deactivated.

Manual stock adjustment

Admin selects stock-in, stock-out, or corrected-count adjustment and supplies a reason.

Service validates that stock cannot become negative and stock-out cannot exceed availability.

Product.stock_quantity and Inventory current/available amounts are kept synchronized.

A movement row, audit event, and possible low/out-of-stock notification are created.

Inventory views

The inventory page supports filtering by product/SKU, category, brand, and stock status; it also shows summary cards, category/status charts, and movement history. Admins see adjustment and reorder controls; analysts have read access and history access.

5. Sales to inventory flow

This is the most important cross-module operational flow.

Admin or Analyst starts a sale and adds one or more product line items.

The service loads each product inside the caller company and combines duplicate requested quantities.

It validates all item availability before applying any deduction, so an unfulfillable multi-item sale fails as a whole.

For each line, total = (unit price x quantity - discount) + tax. Header totals are calculated from the lines.

The service generates a company-scoped invoice such as INV-YYYY-000001 and persists the sale plus SaleItems.

Stock is reduced, a SALE inventory movement and audit log are recorded, and threshold crossings create notifications.

Updating or deleting a sale restores prior stock first, then reapplies the edited sale if appropriate.

Sales filters and reporting dimensions

Sales can be searched by invoice, customer, or product and filtered by dates, category, channel, and payment method. Supported channels are retail store, online store, and marketplace; payment methods are cash, card, UPI, and bank transfer.

6. Analytics, notifications, and exports

Analytics dashboard

Company Admins and Analysts can apply date, product, category, brand, sales-channel, and payment-method filters. Dates, revenue, and sales trends support daily, weekly, or monthly granularity.

- **KPIs: total revenue, orders, products sold, average order value, inventory value,** low stock count, out of stock count, and category count.
- **Visuals: revenue/sales trends, top products/categories, payment/channel mix,** inventory distribution, stock status, inventory value, and low/out-of-stock lists.
- **Drill-downs: clicking KPIs returns underlying sales/inventory records; categories** drill to products; products drill to transactions.
- **Exports: /analytics/export produces CSV or PDF from the currently selected filters** and granularity.

Notifications and audit

Inventory threshold transitions produce LOW_STOCK or OUT_OF_STOCK notifications. Users can fetch notifications and mark them read. Authentication, catalog, sale, stock, dashboard filter/view, and report-export actions are written to the audit log.

API modules

/auth, /users, /categories, /products, /sales, /inventory, /notifications, and /analytics are registered in FastAPI main.py. The health endpoint is /health.

7. Project navigation and operating notes

Where to look in the repository

- **frontend/src/App.tsx: route map. api/client.ts: token injection and refresh.** context/AuthContext.tsx: session state.
- **frontend/src/pages/: feature screens. AnalyticsDashboardPage uses charts, filters,** drill-downs, refresh, and exports.
- **backend/app/main.py: application startup, middleware, models, and router** registration.
- **backend/app/api/: endpoint boundaries. services/: business logic. repositories/:** queries. models/: database relationships.

Configuration and production considerations

- **Backend reads DATABASE_URL, SECRET_KEY, token lifetimes, and FRONTEND_ORIGIN from** environment variables. The code has development defaults; production should provide secure values.
- **The backend calls Base.metadata.create_all at startup. The source notes that** Alembic migrations should be used for production schema changes.
- **CORS currently permits the configured frontend origin with credentials. Keep the** origin explicit in deployed environments.

End-to-end summary

RetailPulse begins with company onboarding and JWT-secured access. Admins establish categories and products; analysts and admins capture sales. Each sale updates stock, records movement/audit evidence, and may create a stock alert. Analytics then aggregates the same company-scoped sales and inventory data into actionable KPIs, drill-downs, and exports.